MATERIAL COMPLEMENTAR / GUIA TÉCNICO EP

Polars e Análise de Dados

Expressões colunares, modelagem de tabelas, agregações, junções, Lazy API e validação analítica.

Propósito — servir como material de estudo aprofundado e referência rápida, com exemplos originais, atividades e integração com os notebooks.

Como utilizar — leia os conceitos, reproduza os exemplos no ambiente indicado e compare as atividades com o notebook correspondente. A consulta rápida aparece nas seções finais.

ROTEIRO DO DOCUMENTO

1. Modelo de execução e papel do Polars
2. Selecionar, filtrar e transformar
3. Tipos, casts e tratamento de strings
4. Ausência, nulidade e qualidade de dados
5. Agregações, group_by e interpretação
6. Ordenação, unicidade e inspeção
7. Joins: integrar tabelas sem inflar métricas
8. Datas, séries temporais e granularidade
9. Lazy API: descrever antes de executar
10. Scan de arquivos e formatos de armazenamento
11. Integração com Pandas e ecossistema Python
12. Desenho de pipeline e validação
13. Estudo de caso: receita por região e categoria
14. Erros e decisões de desempenho
15. Consulta rápida: expressões e operações
16. Oficina prática e exercícios
17. Aprofundamento: expressão, contexto e semântica
18. Aprofundamento: cardinalidade, joins e testes de negócio
19. Aprofundamento: avaliação de qualidade e custo de execução

20. Aprofundamento: relatório e comunicação dos indicadores
21. Integração, referências e próximos estudos

1. Modelo de execução e papel do Polars

Polars é uma biblioteca de manipulação de dados tabulares orientada a expressões, com execução eficiente e suporte a avaliações imediatas (**eager**) e adiadas (**lazy**). Em Data Science, seu valor não é apenas desempenho: operações declarativas tornam transformações mais explícitas, auditáveis e reutilizáveis.

Um DataFrame organiza dados em colunas tipadas. Expressões como `pl.col('receita') * 0.9` descrevem cálculos que a biblioteca executa em contexto de seleção, transformação ou agregação. Compare essa abordagem com loops linha a linha: expressões tendem a comunicar melhor a intenção analítica.

Primeiro exemplo reproduzível

```
import polars as pl

vendas = pl.DataFrame({
    'regiao': ['Norte', 'Sul', 'Norte', 'Sul'],
    'categoria': ['A', 'A', 'B', 'B'],
    'unidades': [2, 4, 3, 1],
    'preco': [15.0, 10.0, 8.0, 25.0],
})
print(vendas.schema)
print(vendas.shape)
```

O esquema (`schema`) explicita tipos das colunas; a forma (`shape`) informa linhas e colunas. A interpretação correta de uma métrica depende também das unidades e do significado de cada registro.

2. Selecionar, filtrar e transformar

`select` define colunas de saída; `with_columns` acrescenta ou substitui colunas preservando as demais; `filter` mantém linhas que atendem a uma condição. Organize operações em pipelines legíveis, com parênteses e nomes intermediários significativos.

```
resumo = (
    vendas
    .with_columns((pl.col('unidades') * pl.col('preco'))
                  .alias('receita'))
    .filter(pl.col('unidades') >= 2)
    .select('regiao', 'categoria', 'receita')
)
print(resumo)
```

Distinção importante: `select` reduz o conjunto de colunas; `with_columns` transforma/insere preservando outras colunas. Uma operação pode ser encadeada sem atribuir manualmente cada passo, mas durante a aprendizagem convém inspecionar resultados intermediários.

3. Tipos, casts e tratamento de strings

Em tabelas reais, a mesma coluna pode chegar como texto, número ou data. Faça conversões explicitamente e inspecione falhas de parsing. Em uma coluna de preços, por exemplo, a convenção decimal e a existência de separadores de milhares interferem no resultado.

```
base = pl.DataFrame({'codigo':[' a1 ', ' B2 '],
                     'quantidade_txt':['3', '7']})
limpa = base.with_columns(
    pl.col('codigo').str.strip_chars().str.to_uppercase(),
    pl.col('quantidade_txt').cast(pl.Int64).alias('quantidade'),
)
print(limpa)
```

Use conversão `strict=False` apenas quando for aceitável mapear valores inválidos para nulos. Não deixe falhas de qualidade silenciosas passarem para o relatório final.

4. Ausência, nulidade e qualidade de dados

`null` representa ausência de valor; `NaN` é um valor especial de ponto flutuante, normalmente associado a resultados numéricos indefinidos. Eles não são intercambiáveis. Antes de preencher ou descartar observações, descubra **por que** estão ausentes.

```
medidas = pl.DataFrame({'grupo':['A','A','B'],
                        'valor':[10.0, None, 5.0]})
print(medidas.null_count())
limpas = medidas.with_columns(
    pl.col('valor').fill_null(0).alias('valor_preenchido')
)
print(limpas)
```

Preencher nulos por zero pode alterar médias e totais. Em análises observacionais, registre a regra aplicada e avalie se a ausência indica “não medido”, “não se aplica” ou efetivamente valor zero.

5. Agregações, group_by e interpretação

Agrupar dados é transformar observações detalhadas em indicadores. Escolha agregações que respondam à pergunta e indique denominadores. Soma de receita, média de preço e contagem de transações possuem interpretações distintas.

```
indicadores = (
    vendas
    .with_columns((pl.col('unidades') * pl.col('preco'))
                  .alias('receita'))
    .group_by('regiao')
    .agg(
        pl.len().alias('linhas'),
        pl.col('unidades').sum().alias('unidades'),
        pl.col('receita').sum().alias('receita'),
    )
    .sort('regiao')
)
print(indicadores)
```

`pl.len()` conta linhas do grupo; não é a soma de unidades vendidas. A escolha do indicador depende do que cada linha representa na tabela original.

6. Ordenação, unicidade e inspeção

`sort` organiza linhas; `unique` remove duplicidades segundo critérios; `head` e `sample` ajudam a explorar. Antes de deduplicar, defina a chave de negócio e avalie se registros repetidos são eventos válidos.

Operação	Objetivo	Cuidados
`.head(5)`	Inspeção rápida	Amostra inicial pode não representar o todo
`.sort('receita')`	Ordenar	Confirme direção e tratamento de nulos
`.unique(subset=[...])`	Remover duplicatas	Defina chave e critério
`.filter(...)`	Selecionar linhas	Verifique filtros que eliminam tudo
`.select(...)`	Selecionar colunas	Preserve campos necessários à auditoria
`.null_count()`	Quantificar nulos	Não identifica automaticamente a causa

7. Joins: integrar tabelas sem inflar métricas

Uma junção combina linhas a partir de chaves. Se uma chave aparece repetida dos dois lados, a multiplicação de linhas pode inflar totais. Antes da junção, inspecione cardinalidade e unicidade de cada tabela.

```
produtos = pl.DataFrame({'produto':['P1','P2'],
                         'categoria':['A','B']})
pedidos = pl.DataFrame({'produto':['P1','P1','P2'],
                        'qtd':[2,1,3]})
unidos = pedidos.join(produtos, on='produto', how='left')
assert unidos.height == pedidos.height
print(unidos)
```

O `assert` funciona para o exemplo porque a dimensão `produtos` tem chaves únicas. Para dados reais, teste primeiro a unicidade e investigue chaves sem correspondência.

8. Datas, séries temporais e granularidade

Datas precisam de tipo e granularidade explícitos. Ano, mês, dia e instante não são equivalentes. Ao agrupar por mês, compare intervalos completos e lembre-se de que zonas horárias podem afetar a definição de “dia”.

```
movimentos = pl.DataFrame({
    'data_txt':['2026-01-04','2026-01-20','2026-02-02'],
    'valor':[10,15,18],
})
mensal = (
    movimentos
    .with_columns(pl.col('data_txt').str.to_date('%Y-%m-%d')
                  .alias('data'))
    .with_columns(pl.col('data').dt.truncate('1mo').alias('mes'))
    .group_by('mes').agg(pl.col('valor').sum().alias('total'))
    .sort('mes')
)
print(mensal)
```

Os valores e datas são inventados apenas para o exercício. Na publicação, dados sintéticos devem ser claramente diferenciados de evidências reais.

9. Lazy API: descrever antes de executar

A API eager executa cada transformação imediatamente; a API lazy constrói um plano de consulta, que só é materializado quando se chama `collect` ou outra operação terminal. Isso permite otimizações como mover filtros e selecionar apenas colunas necessárias mais cedo no pipeline.

```
consulta = (
    vendas.lazy()
    .filter(pl.col('unidades') > 1)
    .with_columns((pl.col('unidades') * pl.col('preco'))
                  .alias('receita'))
    .group_by('regiao')
    .agg(pl.col('receita').sum().alias('receita_total'))
)
print(consulta.explain())
resultado = consulta.collect()
print(resultado)
```

O plano de consulta é uma explicação da estratégia escolhida pelo otimizador; detalhes mudam entre ambientes. Não interprete o plano como garantia de desempenho sem medições.

10. Scan de arquivos e formatos de armazenamento

`read_csv` carrega um CSV diretamente; `scan_csv` cria uma consulta lazy sobre o arquivo. `read_parquet` e `scan_parquet` são alternativas para dados colunares com esquema mais explícito. Para pipelines extensos, começar pelo `scan_*` pode permitir reduzir I/O.

```
from pathlib import Path
import polars as pl

pasta = Path('outputs')
pasta.mkdir(exist_ok=True)
arquivo = pasta / 'vendas_exemplo.parquet'
vendas.write_parquet(arquivo)
consulta = pl.scan_parquet(arquivo)
print(consulta.select(pl.len().alias('linhas')).collect())
```

CSV é intercambiável e legível, mas os tipos precisam ser inferidos ou especificados. Parquet é compacto e tipado, mas não é editável como texto. Preserve o schema e valide transformações após a leitura.

11. Integração com Pandas e ecossistema Python

Pandas e Polars possuem escolhas diferentes de API, índices e execução. Evite transpor expressões mecanicamente: compare a semântica e os tipos antes de declarar equivalência.

```
import pandas as pd
import polars as pl

pdf = pd.DataFrame({'grupo':['A','A','B'], 'valor':[1,2,3]})
pol = pl.from_pandas(pdf)
pol_res = pol.group_by('grupo').agg(pl.col('valor').sum())
print(pol_res.sort('grupo'))
```

Conversões entre bibliotecas podem depender de pacotes opcionais e gerar cópias em memória. Para grandes dados, avalie custos de interoperabilidade antes de converter repetidamente entre representações.

12. Desenho de pipeline e validação

Uma análise confiável tem etapas distinguíveis: carga, checagem de esquema, limpeza, transformações, agregações, validação e exportação. Valide invariantes que expressem regras reais do domínio.

```
base = vendas.with_columns(
    (pl.col('unidades') * pl.col('preco')).alias('receita')
)
assert base.height == vendas.height
assert base.select(pl.col('receita').is_null().sum()).item() == 0
assert base.select(pl.col('receita').sum()).item() >= 0
```

As verificações acima fazem sentido somente para preços e quantidades não negativos. Em operações com devoluções, descontos e estornos, a regra precisa ser reformulada. Um teste só é útil quando corresponde a uma hipótese válida.

13. Estudo de caso: receita por região e categoria

Pergunta: como se distribui a receita entre regiões e categorias de produto? O dataset sintético possui região, categoria, unidades e preço. Receita por linha é unidades multiplicadas por preço.

```
estudo = (
    vendas.lazy()
    .with_columns((pl.col('unidades') * pl.col('preco'))
                  .alias('receita'))
    .group_by('regiao', 'categoria')
    .agg(pl.col('receita').sum().alias('receita_total'))
    .sort('regiao', 'categoria')
    .collect()
)
print(estudo)
```

Interpretação: o agrupamento soma receitas das linhas em cada combinação de região e categoria. Não representa margem de lucro; para isso, seriam necessários dados de custos e outras definições contábeis. Sempre indique período, filtros e origem dos registros.

Acompanhar outputs gerados pelo notebook

No acervo, o notebook correspondente também demonstra CSV e Parquet em `outputs/`, com resumos de receita e unidades. Esses arquivos pequenos são exemplos didáticos, não fontes externas reais. Os nomes de outputs devem ser conferidos com o notebook e mantidos sem dados sensíveis.

14. Erros e decisões de desempenho

Sintoma	Possível causa	Ação recomendada
Erro de tipo	Parsing ou cast inadequado	Inspecionar `schema` e valores
Total inflado após join	Chave repetida	Testar cardinalidade das dimensões
Resultado nulo inesperado	Dados faltantes	`null_count` e regra de preenchimento
Pipeline lento	Leitura e transformações desnecessárias	Usar `scan_*`, `select` e filtros cedo
Falha de memória	Materialização excessiva	Reduzir projeção, avaliar lazy/streaming
Ordem diferente do esperado	Agregação sem ordenação	`sort` explícito no resultado

A otimização depende de dados, máquina e operações. Compare estratégias com medições que controlem cache, tamanho de entrada e equivalência semântica.

15. Consulta rápida: expressões e operações

Tarefa	Padrão
Coluna por nome	`pl.col('x')`
Constante em expressão	`pl.lit(5)`
Seleção	`.select('x', 'y')`
Nova coluna	`.with_columns(expr.alias('novo'))`
Filtro	`.filter(pl.col('x') > 0)`
Agregação	`.group_by('grupo').agg(pl.col('x').sum())`
Ordenação	`.sort('grupo')`
Join	`.join(outro, on='chave', how='left')`
Diagnóstico de nulos	`.null_count()`
Consulta lazy	`.lazy()` / `pl.scan_csv(...)`
Plano	`.explain()`
Execução	`.collect()`

16. Oficina prática e exercícios

1. Reproduza o conjunto sintético e acrescente uma linha com nulo em `preco`.
2. Conte nulos e proponha uma regra justificada para tratamento.
3. Calcule receitas por região e por categoria, conferindo os totais.
4. Implemente uma dimensão de produtos e teste cardinalidade antes do join.
5. Compare um pipeline eager e lazy; inspecione o plano com `explain`.
6. Escreva CSV e Parquet e confira quais tipos são recuperados.
7. Registre limitações do indicador receita e possíveis vieses da amostra.

Critérios de aceite: o pipeline termina sem exceções; transformações são reproduzíveis; agregações têm interpretação; os testes correspondem a regras reais; outputs não contêm informações sensíveis.

17. Aprofundamento: expressão, contexto e semântica

Uma expressão Polars descreve uma transformação, mas seu significado depende do **contexto** no qual é avaliada. Em `select`, a operação normalmente cria a tabela de saída; em `with_columns`, preserva colunas existentes; e em `group_by(...).agg(...)`, calcula resultados por grupo. Reutilizar a mesma expressão em contextos diferentes pode alterar a forma e a interpretação do resultado.

```
import polars as pl

df = pl.DataFrame({'setor':['X','X','Y'],
                   'valor':[10,20,40]})
print(df.select(pl.col('valor').sum().alias('total')))
print(df.with_columns(pl.col('valor').mean().alias('media_global')))
print(df.group_by('setor').agg(pl.col('valor').mean()))
```

Antes de interpretar as saídas, pergunte qual a granularidade: uma linha para o dataset inteiro, uma linha por registro ou uma linha por grupo? Esse raciocínio previne erros de relatório que não são detectados por `pip check` nem pelo esquema do DataFrame.

18. Aprofundamento: cardinalidade, joins e testes de negócio

Imagine uma tabela de pedidos com uma linha por item e uma dimensão de produtos. Uma junção do tipo left deve preservar a contagem de linhas da tabela de pedidos **somente quando** a chave do lado direito é única. Se ela não for, o total da receita pode aumentar artificialmente.

```
pedido = pl.DataFrame({'sku':['A','A','B'], 'receita':[10,20,15]})
dimensao = pl.DataFrame({'sku':['A','B'], 'grupo':['G1','G2']})
assert dimensao.select(pl.col('sku').n_unique()).item() == dimensao.height
base = pedido.join(dimensao, on='sku', how='left')
assert base.height == pedido.height
assert base['receita'].sum() == pedido['receita'].sum()
```

Uma validação completa também deve examinar chaves nulas, registros sem correspondência, relações um-para-muitos esperadas e possíveis mudanças ao longo do tempo. O exemplo acima ilustra o caso simples, não substitui validações de produção.

19. Aprofundamento: avaliação de qualidade e custo de execução

Otimização deve começar por **correção de resultado**. Em seguida, reduza processamento desnecessário, consulte o plano lazy e compare alternativas. `scan_parquet` pode evitar a materialização de colunas irrelevantes; `filter` e `select` permitem ao otimizador empurrar operações para a leitura quando viável.

Pergunta	Verificação prática
O resultado é correto?	Compare totais e invariantes com uma amostra pequena
A consulta lê dados demais?	Verifique projeção e filtros no plano com `explain()`
A junção multiplicou linhas?	Compare contagens antes/depois e unicidade das chaves
Há nulos inesperados?	Inspecione `null_count()` e falhas de cast
O resultado é estável?	Repita a execução e ordene quando a ordem for relevante
O formato é apropriado?	Compare esquema e tamanho de CSV e Parquet

Em análises de dados, desempenho sem rastreabilidade pode ocultar erro. Registre a transformação, as hipóteses e as checagens de qualidade junto aos resultados.

20. Aprofundamento: relatório e comunicação dos indicadores

A métrica “receita por região” depende da definição de receita e da unidade de registro. O número de linhas pode representar pedidos, itens, alterações de pedido ou pagamentos; a agregação só faz sentido quando essa granularidade está clara.

Exemplo de interpretação responsável: “A região Norte concentrou maior receita na amostra sintética, calculada como unidades multiplicadas por preço. O resultado não pode ser generalizado para vendas reais, pois os dados foram construídos apenas para demonstrar as operações.”

Para apresentar resultados, disponibilize tabela de resumo, definição da métrica, filtros, qualidade dos dados e limitações. Se houver visualização, escolha uma escala legível e não esconda categorias relevantes. Uma boa análise termina com uma explicação compreensível, não com a última linha de código.

21. Integração, referências e próximos estudos

Notebook correspondente: `04_Polars_e_Analise_de_Dados.ipynb`. Compare os exemplos aqui com seus exercícios sobre Lazy API, transformações, qualidade e estudo de caso. Os outputs do notebook permanecem no diretório `outputs/`.

- Polars user guide: https://docs.pola.rs/user-guide/
- Polars expressions: https://docs.pola.rs/user-guide/expressions/
- Polars lazy API: https://docs.pola.rs/user-guide/concepts/lazy-api/
- Pandas documentation: https://pandas.pydata.org/docs/
- Parquet format: https://parquet.apache.org/docs/

Documento autoral. A seleção dos temas utiliza a prática acadêmica e referências rápidas como orientação, sem reproduzir tabelas ou redação de terceiros.